

Software Design Principles Lab Reports

Project: CodeHealth AI — An AI-Powered Codebase Health Assistant

Name: Rajat (Team Lead)

Roll Number: 2400290130137

Name: Namrata (Team Member)

Roll Number: 2400290130118

Domain: Developer Tools / AI-Assisted Software Engineering

Covers: Experiment 2 (Problem Identification & Feasibility), Experiment 3 (Requirement Elicitation & User Stories), Experiment 4 (Software Requirements Specification)

Experiment 2: Problem Identification and Feasibility Analysis

Objective: Identify the problem addressed by the selected software project and prepare its problem statement, objectives, scope, and technical and operational feasibility analysis.

Objectives

- Identify the real-world problem addressed by the software project.
- Prepare a clear problem statement.
- Define project objectives and scope.
- Perform technical and operational feasibility analysis.
- Evaluate the viability of the proposed solution.

Introduction

Problem identification is the first step in software engineering. A well-defined problem statement and feasibility study ensure that the proposed software solution is practical, achievable, and aligned with stakeholder needs before development begins.

Selected Project

Project Title	CodeHealth AI
Domain	Developer Tools / AI-Assisted Software Engineering / DevOps

Problem Statement

As software projects grow over time, codebases naturally accumulate technical debt in the form of overly complex functions, duplicated logic, unused (dead) code, and outdated dependencies. Developers are often aware that a project has quality problems but lack an objective way to know which issues matter most. Manually reviewing large repositories file-by-file is slow, inconsistent, and easy to postpone indefinitely. Existing static-analysis tools (ESLint, SonarQube, etc.) generate long lists of raw warnings without explaining why an issue matters, how severe it is relative to other issues, or how to fix it in practical terms. CodeHealth AI addresses this gap by connecting to a developer’s GitHub repository, analyzing it for common technical-debt categories, producing an overall repository health score out of 100, ranking issues by priority, and using AI to explain the highest-impact issues in plain language with a recommended fix.

Project Objectives

- Allow developers to securely connect a GitHub repository using OAuth.
- Statically analyze JavaScript/TypeScript code for complexity, duplication, dead code, and outdated dependencies.
- Generate a quantitative repository health score (0–100) with a category-wise breakdown.
- Prioritize issues by impact so developers know what to fix first.
- Use AI (via Groq) to generate plain-language explanations and refactoring suggestions for the highest-priority issues, while using predefined templates for minor issues to control cost and latency.
- Keep the application responsive during analysis by running scans asynchronously in a background worker.

Project Scope

In scope (MVP): GitHub OAuth login, manual repository selection and scan trigger, static analysis of JavaScript/TypeScript repositories using Tree-sitter, detection of high complexity, duplicate code, possible unused exports, and outdated dependencies, health-score computation, priority-ranked issue list, AI-generated

explanations for top issues via Groq, predefined recommendations for minor issues, and a React/Tailwind dashboard with repository details view.

Out of scope (future phases): Automatic scans via GitHub webhooks on every push, historical health trend tracking, multi-user team dashboards, support for languages beyond JavaScript/TypeScript, and AI-generated automatic refactoring pull requests.

Technical Feasibility Analysis

Factor	Assessment	Remarks
Technology	Feasible	Node.js/Express, React, Tailwind CSS, MongoDB, Redis, BullMQ, Tree-sitter, Groq API, and GitHub REST/OAuth APIs are all mature, well-documented, and freely usable for an MVP.
Hardware	Feasible	A standard cloud VM or free-tier hosting (e.g., Render/Railway) with Redis and MongoDB Atlas is sufficient for MVP-scale repository analysis.
Software Tools	Feasible	GitHub OAuth apps, Tree-sitter parsers, BullMQ job queues, and Groq's free/low-cost LLM tier are readily available.
Security	Feasible	GitHub OAuth with scoped tokens, JWT-based sessions, and encrypted token storage can be implemented using established libraries.
Scalability	Feasible	Separating the API from the analysis worker via a Redis/BullMQ queue allows horizontal scaling of workers independently of the web tier as repository volume grows.

Operational Feasibility Analysis

Factor	Assessment	Remarks
User Acceptance	High	Developers already use GitHub daily; a one-click OAuth login and familiar dashboard lowers adoption friction.
Ease of Use	High	The workflow (login → select repo → run scan → view prioritized issues) requires no training or documentation.
Availability	High	The asynchronous worker architecture keeps the dashboard responsive even during long-running scans, ensuring consistent availability.
Maintainability	High	Modular static-analysis rules, a decoupled worker process, and clear MongoDB schemas make the codebase easy to extend with new checks or languages.
Business Benefit	High	Converts an overwhelming list of raw warnings into a short, prioritized action plan, saving developer time and reducing long-term maintenance cost.

Analysis

The feasibility study indicates that CodeHealth AI is both technically and operationally viable for an MVP scope. All required technologies are open-source or offer generous free tiers, and the background-worker pattern (Redis + BullMQ) directly solves the key technical risk of long-running repository scans blocking the user interface.

Result

The problem statement, objectives, project scope, and technical and operational feasibility analysis for CodeHealth AI were successfully prepared, confirming that the project is a viable candidate for implementation.

Experiment 3: Stakeholder Identification, Requirement Elicitation and User Stories

Objective: Identify stakeholders and perform requirement elicitation for CodeHealth AI using suitable techniques (interviews, questionnaires, observation, user personas); prepare user stories and a Requirement Gathering Report.

Objectives

- Identify key stakeholders of CodeHealth AI.
- Apply suitable requirement elicitation techniques.
- Gather functional and non-functional requirements.
- Prepare user stories with acceptance criteria.
- Document the Requirement Gathering Report.

Introduction

Requirement elicitation is the process of discovering stakeholder needs and expectations. The quality of software depends on how accurately these requirements are captured. Common elicitation techniques include interviews, questionnaires, observation, workshops, and user personas.

Selected Project

Project Title	CodeHealth AI
Domain	Developer Tools / AI-Assisted Software Engineering

Stakeholder Identification

Stakeholder	Role	Requirements
Individual Developer	Connects a GitHub repo and runs scans to check code quality	Fast, accurate analysis and clear, actionable recommendations
Engineering/Team Lead	Reviews repository health to guide refactoring priorities	Reliable health scores and issue prioritization across projects
GitHub (Platform/API)	Supplies repository data via OAuth and REST APIs	Correct API usage, rate-limit compliance, secure token handling
Groq AI Service	Generates natural-language explanations for high-priority issues	Well-formed prompts, predictable request volume, graceful failure handling
Background Worker System	Executes long-running scans without blocking the API	Reliable job queueing, retries on failure, and progress reporting

Requirement Elicitation Techniques

Technique	Purpose	Outcome
Interview (self/peer developers)	Discuss pain points with maintaining long-lived codebases	Detailed functional requirements on complexity, duplication, and dead-code detection
Questionnaire	Collect feedback from multiple student/junior developers	User expectations around scan speed, clarity of explanations, and trust in AI suggestions
Observation	Observe how developers currently use ESLint/SonarQube-style tools	Usability and workflow requirements for the dashboard and issue list
User Persona	Model representative users of the platform	User-centred requirements for prioritization and explanation depth

Sample Requirement Gathering

Interview Findings:

- Developers want to log in with their existing GitHub account, without creating a new password.
- Scans on large repositories should not freeze or block the dashboard.
- Raw linter warnings are not enough — developers want to know why an issue matters and what to fix first.
- AI explanations should be reserved for the most impactful issues to avoid unnecessary cost and delay.

Observation Findings:

- Developers currently switch between multiple tools (ESLint, npm audit, manual code review) to get a full picture of code health.
- A single consolidated health score is easier to communicate to non-technical stakeholders (e.g., project managers).

Questionnaire Findings:

- Users expect scan results within a reasonable time even for medium-sized repositories.
- Security of GitHub tokens and repository code is a top concern.

User Personas

Persona	Age	Goal	Pain Point
Student Developer	20	Understand and clean up a college/personal project before submission	Doesn't know which part of the code is actually a priority to fix
Junior Full-Stack Developer	24	Quickly assess an inherited codebase at a new job	Overwhelmed by hundreds of raw linter warnings with no ranking
Engineering/Team Lead	32	Track technical debt across the team's repositories over time	No single score or dashboard summarizing repository health

User Stories

ID	User Story	Acceptance Criteria
US1	As a developer, I want to log in using my GitHub account so that I can access the platform without creating a new password.	GitHub OAuth login succeeds and issues a valid session; feature passes functional testing.
US2	As a developer, I want to select one of my repositories and click 'Run Scan' so that the system analyzes its code health.	Selected repository is queued for a scan job and status is visible on the dashboard.
US3	As a developer, I want to see an overall health score out of 100 so that I can quickly judge my repository's quality.	Health score and category breakdown display correctly after scan completion.
US4	As a developer, I want the most important issues shown first so that I know what to fix before anything else.	Issues are ranked by priority/impact and the top issues appear at the top of the list.
US5	As a developer, I want a plain-language explanation and fix recommendation for high-priority issues so that I understand the problem without deep static-analysis knowledge.	AI-generated explanation, impact statement, and refactor suggestion display for top-ranked issues.
US6	As a developer, I want the dashboard to stay responsive while my repository is being analyzed so that I am not blocked from using the app.	Scan runs asynchronously in a background worker; dashboard shows live scan progress without freezing.

Requirement Gathering Report

The elicitation process identified GitHub authentication, repository selection, asynchronous scanning, technical-debt detection (complexity, duplication, dead code, outdated dependencies), health scoring, issue prioritization,

and AI-generated explanations as the primary requirements for CodeHealth AI. Stakeholder feedback emphasized speed, trustworthy prioritization, plain-language explanations, and a responsive interface even during long-running scans.

Observation

Using multiple elicitation techniques (interviews, observation, questionnaires, personas) improved requirement completeness and reduced ambiguity, particularly around how much explanation developers need for each issue and how the health score should be broken down by category.

Result

Stakeholders were identified, requirements were elicited using suitable techniques, user stories were prepared, and the Requirement Gathering Report for CodeHealth AI was successfully completed.

Experiment 4: Software Requirements Specification (SRS)

Objective: Prepare a complete Software Requirements Specification document for CodeHealth AI following the standard SRS structure (Wiegiers-style template), clearly and unambiguously defining the system's functional and non-functional requirements.

Document Information

Project	CodeHealth AI
Document	Software Requirements Specification, Version 1.0
Prepared by	Rajat (Team Lead), Namrata (Team Member)
Domain	Developer Tools / AI-Assisted Software Engineering

1. Introduction

- 1.1 Purpose:** This SRS specifies the functional and non-functional requirements for CodeHealth AI, an AI-powered codebase health assistant for JavaScript/TypeScript GitHub repositories, Version 1.0 (MVP). It covers the manual-scan MVP scope only; automatic webhook-triggered scans, multi-language support, and team dashboards are explicitly deferred to future releases.
- 1.2 Document Conventions:** Functional requirements are labeled REQ-x.x and are grouped under the system feature they belong to. Priority is expressed as High, Medium, or Low; higher-level feature priority is inherited by its detailed requirements unless stated otherwise.
- 1.3 Intended Audience and Reading Suggestions:** This document is intended for the project team (architect and implementers), academic reviewers/evaluators, and any future collaborators. Readers should begin with Section 2 (Overall Description) for context, then proceed to Section 4 (System Features) for detailed functional requirements relevant to frontend, backend, or worker implementation.
- 1.4 Product Scope:** CodeHealth AI helps developers understand and prioritize technical debt in their codebases. It connects to GitHub, statically analyzes JavaScript/TypeScript repositories for complexity, duplication, dead code, and outdated dependencies, computes an overall health score, and uses AI to explain and recommend fixes for the highest-priority issues. The MVP goal is to convert a large, unreviewed codebase into a short, prioritized, explainable action list.
- 1.5 References:** GitHub REST API and OAuth documentation; Tree-sitter parser documentation; BullMQ and Redis documentation; Groq API documentation; MongoDB schema design references.

2. Overall Description

- 2.1 Product Perspective:** CodeHealth AI is a new, self-contained, GitHub-integrated web platform. It is composed of a React/Tailwind frontend, a Node.js/Express API, a separate Node.js worker process, a Redis/BullMQ job queue, and a MongoDB data store, with GitHub and Groq as external service dependencies.
- 2.2 Product Functions:**
- GitHub OAuth-based authentication and repository listing.
 - Manual scan trigger ('Run Scan') for a selected repository.
 - Asynchronous scan execution via a Redis/BullMQ job queue and worker process.
 - Static analysis using Tree-sitter for complexity, duplication, unused exports, and dependency checks.
 - Health-score computation (0–100) with category-wise breakdown.
 - Priority-based issue ranking (impact vs. effort).
 - AI-generated explanation, impact statement, and refactor suggestion for top-priority issues via Groq.
 - Predefined, template-based recommendations for lower-priority issues.

- Dashboard and repository-details views showing score, breakdown, scan progress, and recommendations.

2.3 User Classes and Characteristics: Primary user class: individual developers with basic-to-intermediate technical expertise who own or contribute to a GitHub repository. Secondary user class: team/engineering leads who review health scores of multiple repositories (planned for a future team-dashboard phase, not MVP).

2.4 Operating Environment: Frontend: modern web browsers (Chrome, Edge, Firefox) accessing a React/Tailwind CSS single-page application. Backend: Node.js/Express server and a separate Node.js worker process, deployed on a cloud host with network access to GitHub and Groq APIs. Data stores: MongoDB (results/metadata) and Redis (job queue).

2.5 Design and Implementation Constraints: MVP is restricted to JavaScript and TypeScript repositories only. Analysis must run asynchronously via BullMQ/Redis to avoid blocking API request threads. GitHub API rate limits constrain how frequently and how much repository data can be fetched per scan. Groq API usage is restricted to high-priority issues only, to control cost and latency.

2.6 User Documentation: An in-app onboarding flow (landing page + login instructions) and a project README/architecture document serve as user and technical documentation for the MVP.

2.7 Assumptions and Dependencies: Assumes the user has a GitHub account and grants OAuth read access to at least one repository. Depends on availability and rate limits of the GitHub REST API and the Groq inference API. Depends on Tree-sitter grammars correctly parsing JavaScript/TypeScript syntax.

3. External Interface Requirements

3.1 User Interfaces: Landing page, GitHub login page, dashboard (health score, breakdown, scan progress, top recommendations), and repository details page (file-level issues and AI explanations), all built with React and Tailwind CSS following a consistent, developer-tool-appropriate visual style.

3.2 Hardware Interfaces: None beyond standard client devices (desktop/laptop browsers) and standard server infrastructure; no specialized hardware is required.

3.3 Software Interfaces: GitHub REST API and OAuth (repository listing, file fetching, authentication); Groq API (LLM-based explanation generation); MongoDB (persistent storage of users, repositories, and scan results); Redis/BullMQ (job queue for asynchronous scans).

3.4 Communications Interfaces: HTTPS for all client-server and server-to-external-API communication; JSON as the standard message format for REST endpoints between frontend, backend, and worker-reported results.

4. System Features

4.1 GitHub OAuth Authentication

Description and Priority: High priority.

Stimulus/Response Sequence: User initiates login → redirected to GitHub → authorizes → redirected back with an authorization code → backend exchanges code for a token and creates a session.

Functional Requirements:

- REQ-1.1: The system shall allow a user to log in exclusively via GitHub OAuth.
- REQ-1.2: The system shall securely store the GitHub access token associated with the user session.
- REQ-1.3: The system shall reject requests to protected routes without a valid session.

4.2 Repository Selection and Scan Trigger

Description and Priority: High priority.

Stimulus/Response Sequence: User selects a repository from a list fetched from GitHub → clicks 'Run Scan' → backend creates a scan job and returns a job/scan ID.

Functional Requirements:

- REQ-2.1: The system shall list the user's accessible JavaScript/TypeScript GitHub repositories.
- REQ-2.2: The system shall allow the user to trigger a manual scan for a selected repository.
- REQ-2.3: The system shall reject scan requests for unsupported (non-JS/TS) repositories with a clear message.

4.3 Asynchronous Scan Execution (Queue and Worker)

Description and Priority: High priority.

Stimulus/Response Sequence: API enqueues a scan job in Redis/BullMQ → worker process picks up the job → fetches repository files from GitHub → runs analysis.

Functional Requirements:

- REQ-3.1: The API shall enqueue scan requests as BullMQ jobs instead of processing them synchronously.
- REQ-3.2: The worker shall retry a failed job automatically up to a configured retry limit if a worker crash or transient external-API failure occurs.
- REQ-3.3: The system shall expose scan progress/status so the dashboard can display real-time updates.

4.4 Static Code Analysis

Description and Priority: High priority.

Stimulus/Response Sequence: Worker parses fetched files using Tree-sitter and runs detection modules for complexity, duplication, dead code, and dependencies.

Functional Requirements:

- REQ-4.1: The system shall detect functions/methods exceeding a defined cyclomatic-complexity threshold.
- REQ-4.2: The system shall detect duplicate or near-duplicate code blocks across files.
- REQ-4.3: The system shall flag exports with no detected internal usage as possible unused/dead code.
- REQ-4.4: The system shall check package.json dependencies against latest published versions and flag outdated packages.

4.5 Health Score Calculation

Description and Priority: High priority.

Stimulus/Response Sequence: Worker aggregates category-level findings into a single weighted score once analysis completes.

Functional Requirements:

- REQ-5.1: The system shall compute an overall repository health score on a 0–100 scale.
- REQ-5.2: The system shall provide a category-wise score breakdown (complexity, duplication, dependencies, dead code).
- REQ-5.3: The system shall persist health-score results in MongoDB linked to the repository and scan ID.

4.6 Issue Prioritization and AI Explanation

Description and Priority: High priority.

Stimulus/Response Sequence: System ranks all detected issues by impact/effort; top-ranked issues are sent to Groq for explanation, lower-ranked issues use static templates.

Functional Requirements:

- REQ-6.1: The system shall rank detected issues by priority (impact vs. effort) before display.
- REQ-6.2: The system shall generate an AI explanation, impact statement, and refactor suggestion via Groq for the top-priority issues only.
- REQ-6.3: The system shall use predefined, non-AI recommendation templates for lower-priority issues.
- REQ-6.4: If the Groq API call fails or times out, the system shall fall back to a predefined recommendation for that issue.

4.7 Dashboard and Repository Details View

Description and Priority: Medium priority.

Stimulus/Response Sequence: User views the dashboard after login and drills into a specific repository's scan results.

Functional Requirements:

- REQ-7.1: The dashboard shall display the health score, score breakdown, scan progress, and top recommendations.
- REQ-7.2: The repository details page shall display the full ranked issue list, per-issue severity, and per-issue explanation.
- REQ-7.3: The system shall clearly indicate when a scan is in progress, completed, or failed.

5. Other Nonfunctional Requirements

5.1 Performance Requirements: Dashboard pages shall remain interactive (no blocking UI) regardless of scan duration. A scan job for a medium-sized repository (up to a few hundred JS/TS files) should complete within a few minutes under normal load. Groq API calls shall be limited to top-priority issues to keep AI-related latency and cost bounded.

5.2 Safety Requirements: Not applicable in the traditional physical-safety sense; the primary 'safety' concern is data safety — scans must never modify the user's source code or repository, only read it.

5.3 Security Requirements: GitHub OAuth tokens shall be stored securely (encrypted at rest) and never exposed to the frontend. All API traffic shall use HTTPS. Session management shall use signed/JWT tokens with expiry. Access to a user's repositories and scan results shall be restricted to that authenticated user.

5.4 Software Quality Attributes: Maintainability: modular analysis rules and a decoupled worker allow new checks or languages to be added without touching the API. Reliability: BullMQ retries make scans resilient to transient worker crashes or external API failures. Usability: plain-language AI explanations make results accessible to developers without deep static-analysis expertise. Scalability: worker processes can be scaled independently of the API as scan volume grows.

5.5 Business Rules: Only authenticated GitHub users may trigger scans on repositories they have access to. AI (Groq) explanations are generated only for the highest-priority issues per scan to control operating cost; all other issues use predefined templates.

6. Other Requirements

Database requirements: MongoDB collections for Users, Repositories, and Scans, with indexes on repository ID and commit SHA to support idempotent re-scans. Internationalization and legal requirements are out of scope for the MVP.

Appendix A: Glossary

Term	Definition
Technical debt	Accumulated code-quality issues (complexity, duplication, dead code, outdated dependencies) that make a codebase harder to maintain.
Health score	A 0–100 composite score representing overall repository code quality across all analyzed categories.
BullMQ	A Redis-backed job/queue library used to run repository scans asynchronously in a worker process.
Tree-sitter	An incremental parsing library used to build syntax trees for static code analysis.
Groq	An LLM inference API used to generate plain-language issue explanations and fix suggestions.

Appendix B: Analysis Models

High-level data flow: User → React Frontend → Express API → BullMQ Job (Redis) → Worker Process → GitHub API (fetch files) + Tree-sitter (parse/analyze) → Health Score Engine → Groq API (top-issue explanations) → MongoDB (persist results) → Dashboard (read results).

Appendix C: To Be Determined List

- TBD-1: Exact cyclomatic-complexity threshold values per severity level.
- TBD-2: Exact weighting formula combining category scores into the overall 0–100 health score.
- TBD-3: Rate-limit and retry configuration values for the GitHub and Groq API clients.

Result

A complete Software Requirements Specification for CodeHealth AI, covering introduction, overall description, external interfaces, seven core system features with functional requirements, non-functional requirements, and supporting appendices, was successfully prepared.